

Opendoor Sell (<https://www.opendoor.com>)

(/)

Buy (<https://www.opendoor.com/homes>) More

Blog Home (<https://www.opendoor.com/w/blog>)

Home Buying (<https://www.opendoor.com/w/blog/category/home-buying-tips>)

Home Selling (<https://www.opendoor.com/w/blog/category/home-selling-tips>)

Company News (<https://www.opendoor.com/w/blog/category/company-news>)

Local Insights (<https://www.opendoor.com/w/blog/category/local-insights>)

Technology (<https://www.opendoor.com/w/blog/category/technology>)

Imputing text data using markov random fields

January 7, 2019 — Written by Jackson Gorham

Share this article

 ([https://www.facebook.com/sharer/sharer.php?](https://www.facebook.com/sharer/sharer.php?u=https://www.opendoor.com/w/blog/imputing-text-data-using-markov-random-fields)

[u=https://www.opendoor.com/w/blog/imputing-text-data-using-markov-random-fields](https://www.opendoor.com/w/blog/imputing-text-data-using-markov-random-fields))

 ([https://www.linkedin.com/shareArticle?](https://www.linkedin.com/shareArticle?mini=true&url=https://www.opendoor.com/w/blog/imputing-text-data-using-markov-random-fields)

[mini=true&url=https://www.opendoor.com/w/blog/imputing-text-data-using-markov-random-fields](https://www.opendoor.com/w/blog/imputing-text-data-using-markov-random-fields))

🐦 ([https://twitter.com/home?](https://twitter.com/home?status=https://www.opendoor.com/w/blog/imputing-text-data-using-markov-random-fields)

[status=https://www.opendoor.com/w/blog/imputing-text-data-using-markov-random-fields](https://www.opendoor.com/w/blog/imputing-text-data-using-markov-random-fields))

✉ (<mailto:?&body=https://www.opendoor.com/w/blog/imputing-text-data-using-markov-random-fields>)



"On two occasions I have been asked, 'Pray, Mr. Babbage, if you put into the machine wrong figures, will the right answers come out?'... I am not able rightly to apprehend the kind of confusion of ideas that could provoke such a question." – Charles Babbage

(https://en.wikipedia.org/wiki/Charles_Babbage), Passages from the Life of a Philosopher

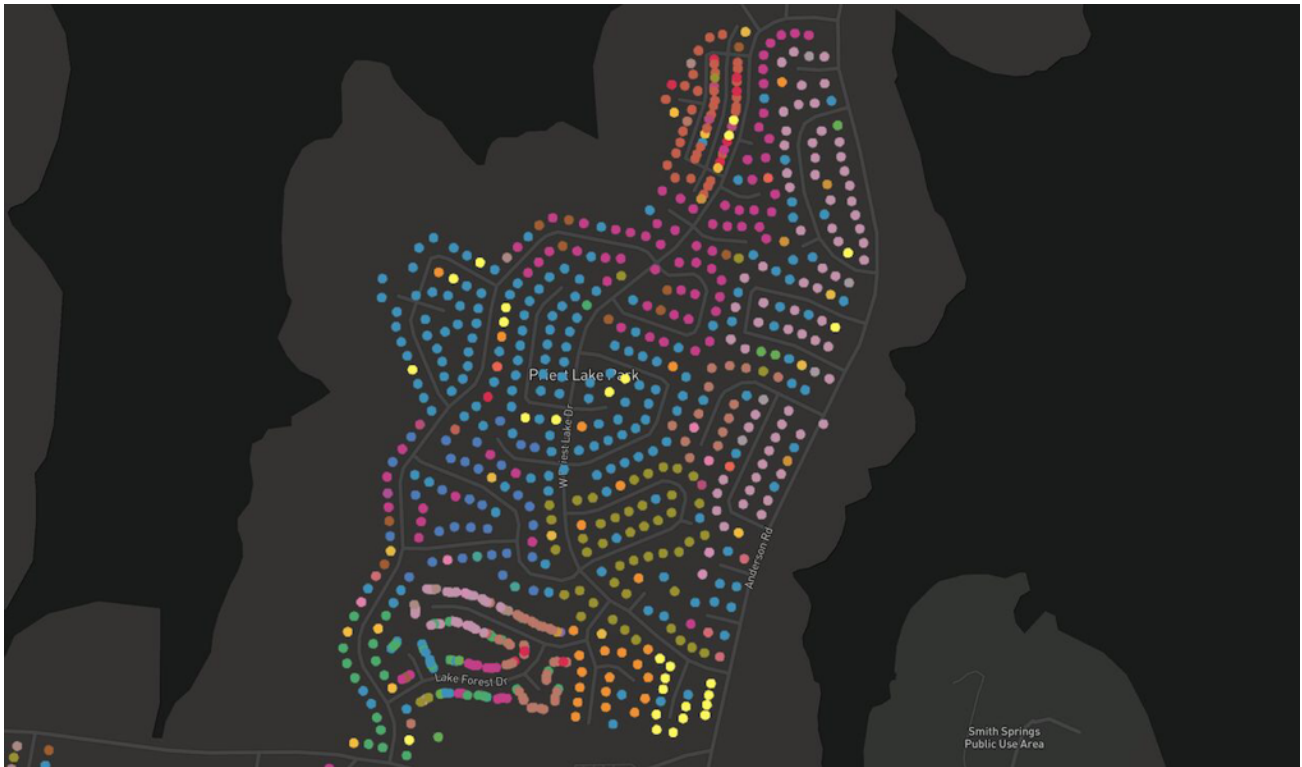
Garbage in, garbage out. This mantra—which has been around since at least the 1950s (<https://www.newspapers.com/newspage/55787725/>)—is paramount in a world where data has become more and more central to computational methods. At Opendoor, we feel the pain of poor data quality acutely: it can easily throw our home valuation models' predictions off hundreds of basis points (https://en.wikipedia.org/wiki/Basis_point).

For many of our models, one of the most important characteristics of a home is the *subdivision*. Roughly speaking, a subdivision consists of a group of homes that were developed generally around the same time, by the same builder, with similar materials and similar architectural styles. Subdivisions thus provide meaningful clusterings of homes in the United States housing market.

Below is an illustration of what great subdivision labels look like for a segment of homes in Phoenix (similar colored homes belong to the same subdivision [1]).



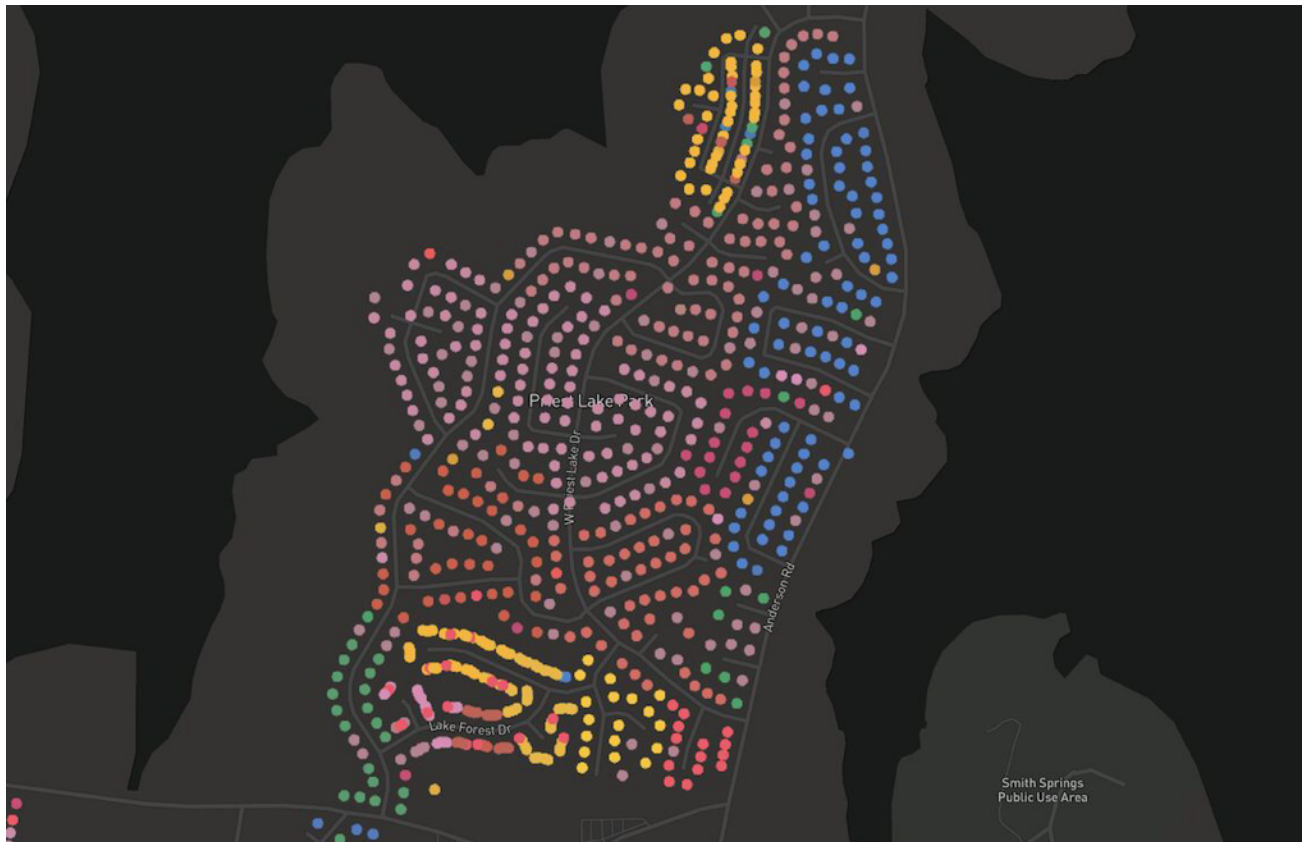
The tessellation of homes above encodes the history of housing development in this area. However, as we will see, the outstanding quality of the labels above is an exception to the rule. Because subdivision labels are text data, the reality across the United States is that the picture of subdivision labels is—to put it lightly—not so neat. The image below illustrates raw subdivision text data for a neighborhood in Nashville. Note the spottiness of the labels.



An 80% Solution: Trial & Error + Finite State Machines

Our CTO Ian emphasizes (<https://medium.com/opendoor-labs/a-few-reflections-on-data-science-9eb0c81c52b6>) incremental progress as a first principle, and in that spirit, we originally searched for an 80% solution to shore up the noisy text data. Many of the mutations in the text data were actually relatively common: abbreviations (e.g. “condo” in lieu of “condominium”), misspellings (e.g. “edition” instead of “addition”), junk words (e.g. “parcel” or “sub”) and numeric spellings (e.g. “seventh” or “VII” instead of ‘7th’). By implementing these standardizations as regular expressions (https://en.wikipedia.org/wiki/Regular_expression) and chaining

them into a massive state machine, we were able to improve the quality of labels. However, after doing this for the same neighborhood pictured above, we arrived at the following labelling:



This is indeed a bit better, but this clearly still lacks the spatial continuity one would expect from accurate subdivision data. Consider the mostly golden lump of homes near the top. Below are some examples of the text data before the regular expressions were applied:

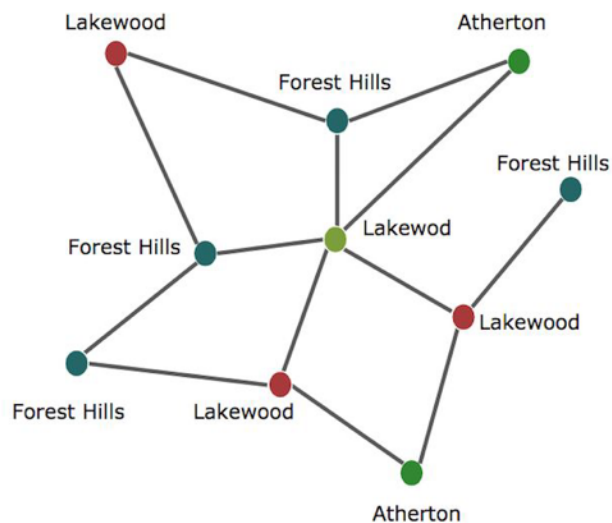
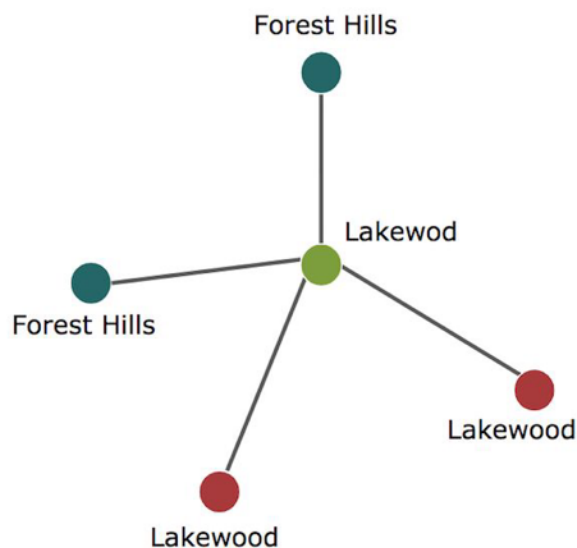
- HERITAGE VILLAGE SEC 02
- HERITAGE VLG/PRIEST LAKE PARK
- HERITAGE VILL / PRIEST LAK

- HERITAGE VLG / PRIEST LAKE CONDO
- HERITAGE VILLAGE SEC2/PRIEST L
- PRIEST LAKE PARK SEC 2 HERITAG

While text standardization helps with some of these, the inconsistent ordering of words and misspellings are highly specific deviations which are difficult to catch by casting a net of regexs. So how can we do better?

An Idea: Leveraging Spatial Continuity of Subdivisions

The regex approach above is highly individualistic: the sanitized subdivision text only depends on the original text. However, we know that in general, subdivisions are spatially continuous. That is, if home A is next door to home B, then it is very likely A and B are in the same subdivision. It isn't always true—like when home A is across a busy street from home B—but this suggests we can use neighbors' information when trying to infer a particular home's subdivision.



Consider the figure on the left. Each circle represents a home, and the text near each circle is the subdivision label we have for each home. If we were trying to infer which subdivision the center home actually belonged to, we would probably surmise it was **Lakewood**. In this case, we see there is a tie for its most common neighbor subdivision (each has count two), but since Lakewood is more like original text we would prefer that label. This suggests a simple scheme for inferring subdivision: for each home, we use the most common label amongst it and its neighbors, and in the event of a tie, we choose the one most similar to the home's own text.

There are a couple problems with this approach. The first—which is not a huge deal—is that we may have to do this many times for the updated labels to propagate through the graph. The second—and more difficult—issue is illustrated in the figure on the right. This illustrates what the surrounding homes might look like if we zoomed out a bit. In this example, we see that

depending on which homes we started with, we would arrive with completely different labelings. This is because the misspelled “**Lakewod**” could propagate through the bottom part of the graph if we started with the red nodes. This suggests we must be a bit more careful to ensure our labels are stable.

One solution to this problem is to use a probabilistic model that scores all labels jointly. The score of each labelling will come from two different components:

- How well the final labels relate to the original text
- How spatially continuous the labels are among neighbors

By constructing a score from these two criteria, we will be able to make headway on learning the entire labelling at once.

A Model: Markov Random Field

How can we articulate this mathematically? One possibility is to borrow the setup from Stuart and Donald Geman’s seminal paper (<https://ieeexplore.ieee.org/abstract/document/4767596>) that articulates a set of Bayesian methods useful for image processing [2]. In their work, they used Markov random fields (https://en.wikipedia.org/wiki/Markov_random_field) as a tool to restore blurry images; their ideas are general enough to port over for our used case here as well.

Suppose someone gives you a graph $G = (V, E)$ with vertices V and edges E , and for each vertex $v \in V$, there is an associated random variable X_v taking values in the set $\mathcal{X}$. Then by borrowing ideas from statistical mechanics, one could posit a joint probability distribution over all outcomes via the unnormalized density function...

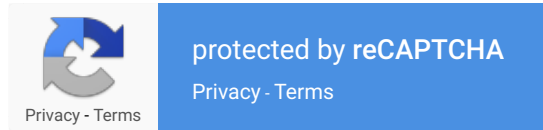
$$p(x_1, \dots, x_{|V|}) = P(X_v = x_v \forall v \in V) \propto \exp \left\{ \sum_{v \in V} \phi_v(x_v) + \sum_{(i,j) \in E} \psi(x_i, x_j) \right\}$$

...for any $x_1, \dots, x_{|V|} \in \mathcal{X}$ and some functions $\{\phi_v\}_{v \in V}, \psi$. Via Bayes' rule (https://en.wikipedia.org/wiki/Bayes%27_theorem), one can show that for any vertex v , if we denote $N_v = \{w \mid (w, v) \in E\}$ to be the neighboring set of v , then $X_v | X_{N_v} = X_v | X_{N_v \cup W}$ where $W \subseteq V \setminus \{v\}$. In words, this says that if you know the values of v 's neighbors X_{N_v} but not X_v , knowing any other nodes values provides no additional information about X_v . This demonstrates p is an example of a Markov random field.

How does this exactly relate to subdivisions? First, notice each housing market induces a graph G : each home corresponds to a vertex, and an edge is drawn between two vertices if and only if they are neighbors. As for the variables $\{X_v\}_{v \in V}$, these represent the "true" latent subdivision labels. If ψ is largest when $x_i = x_j$, this implies that—in general—we expect neighbors to belong to the same subdivision. For example, suppose we chose $\psi(x_i, x_j) = \lambda \cdot \mathbb{1}\{x_i = x_j\}$ for some $\lambda > 0$. This implies that if homes i and j are neighbors, all else equal, we would favor arrangements that set these homes in the same subdivision by a factor of e^λ .

How does the observed subdivision text data fit into this model? For a home corresponding to vertex v , denote t_v as the observed text for that home. Then we can couple t_v to our best guess of X_v through the functions ϕ_v . For instance, suppose $\rho : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}_+$ is some distance between strings, e.g., the edit distance (https://en.wikipedia.org/wiki/Edit_distance). Then we could chose $\phi_v(x) = -\rho(x, t_v)$ for each $v \in V$. If we ignored neighboring subdivision data for the moment, our most likely guess for X_v would be t_v [the current label], the second most likely guesses are those which are edit distance 1 from t_v , and so on.

Note: this model is actually a special case of a conditional random field (https://en.wikipedia.org/wiki/Conditional_random_field), but we can avoid more definitions by treating the true subdivision labels as the only random quantities.



Optimizing the Model

Consider now our unnormalized probability distribution p with the above choices for $\{\phi_v\}_{v \in V}$ and ψ :

$$p(x_1, \dots, x_{|V|}) \propto \exp \left\{ - \sum_{v \in V} \rho(x_v, t_v) + \lambda \sum_{(i,j) \in E} \mathbb{1}\{x_i = x_j\} \right\}$$

We see the first term tries to keep the imputed subdivision labels close to the observed text data, while the second term leverages our knowledge that subdivisions are spatially continuous. The parameter λ allows us to trade off

how much we prefer spatial continuity vs. labels posited by the observed text. This gives us a probabilistic way of trying to mitigate the noise we have in our messy text data!

How can we possibly find which $(x_1, \dots, x_{|V|})$ has the largest probability? This is a challenging optimization problem, as the extraordinarily large number of possible states precludes one from simply computing each unnormalized probability and finding the biggest one. In fact, the problem is NP-hard in general. The technique loopy-belief propagation (https://en.wikipedia.org/wiki/Belief_propagation) is popular for doing inference on many graphical models, but a simpler approach in this case is to rely on a type of Markov chain Monte Carlo (https://en.wikipedia.org/wiki/Markov_chain_Monte_Carlo) called Gibbs sampling (https://en.wikipedia.org/wiki/Gibbs_sampling). The idea is rather than trying to directly maximize the probability distribution above, we pick some starting value, and then we use a technique to iteratively produce guesses that “in the limit” will look like a true sample drawn from the distribution given by p . Once we have enough samples that are approximately drawn from p , we can pick the one with the greatest probability as our optimizer.

How do we produce the chain of guesses? Suppose our current guess is $X^{(k)} = (x_1^{(k)}, \dots, x_{|V|}^{(k)})$. Then for any home v , we can write down the conditional probability of X_v given all the other guesses for homes not v [math exercise left to the reader] as:

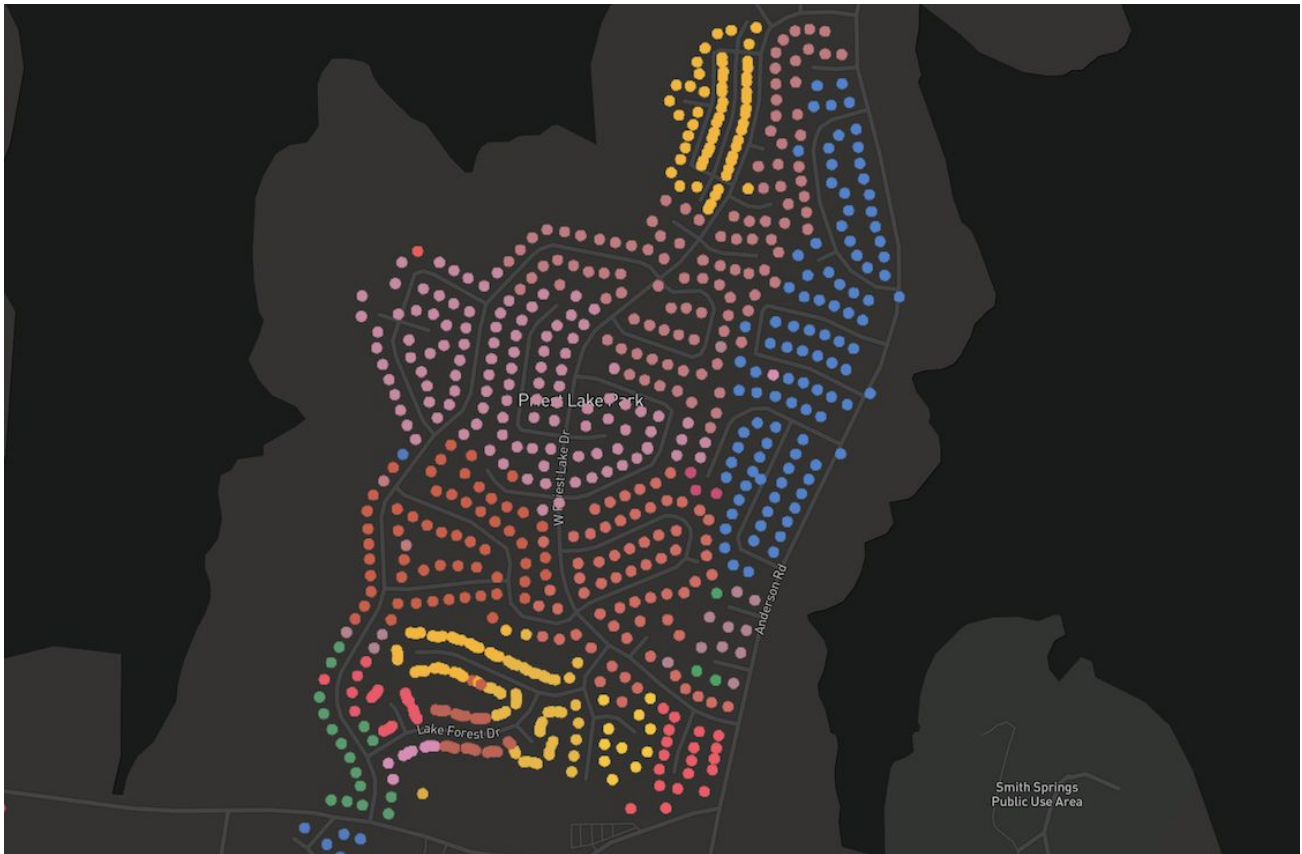
$$P(X_v = x_v | X_w = x_w \forall w \neq v) \propto \exp \{ -\rho(x_v, t_v) + \lambda |\{i \in V \mid x_i = x_v\}| \}$$

This is great! Notice that while this is still an unnormalized distribution, it is a univariate distribution and thus computationally feasible to draw a sample from. Now suppose we produce $X^{(k+1)}$ by doing the following: we choose v uniformly random from V , pick $x_v^{(k+1)}$ by sampling from the univariate distribution above, and fix $x_w^{(k+1)} = x_w^{(k)}$ for all $w \neq v$. Thanks to the beautiful mathematics of Markov chains, if we apply this recursively over and over, then the assignments will begin to look like a sample from p ! This gives us a computationally tractable procedure for optimizing our objective.

How is this implemented? At Opendoor, one of the tools currently at our disposal is PySpark (<https://spark.apache.org/docs/0.9.0/python-programming-guide.html>). By employing GeoSpark (<http://geospark.datasyslab.org/>) on top of this framework, we are able to produce millions of samples using the procedure above. Once we throw out some initial burn-in samples, we can reduce the samples to final single imputation as our best guess for the true subdivision labels.

Does it work in practice?

There are actually quite a few degrees of freedom in the above model and the quality of results can vary—e.g. the choice of λ or ρ are quite important. However, below we show the results of using this algorithm with a simple choice of λ and ρ to produce the imputed subdivision labels:



Science! There are still a few imperfections (in fact this area could have benefitted from generating a longer chain), but overall the picture is much closer to our expectation of subdivision groupings. These techniques have given Opendoor the ability to make its data more than just the sum of its parts.

Interested in joining our team?

From data science to home renovation, we're looking for great people who love solving tough challenges.

Explore our job openings (<https://www.opendoor.com/jobs>)

[1] Technically speaking, the colors specify subdivision phases, which are sub-developments within a single subdivision.

[2] Geman, S., & Geman, D. (1984). Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. IEEE Transactions on pattern analysis and machine intelligence, (6), 721-741.

Also published on Medium

(<https://medium.com/@joseph.gomez/6203e9fc266a>).

Recommended articles



17 essential real estate terms you should know



When is the best time to sell a house? 5 factors to consider



15 expert tips to increase your home value

Join our newsletter!

Get the latest real estate tips, trends & insights delivered straight to your inbox.

Enter your email

Subscribe

 Subscribe to our RSS Feed (<https://www.opendoor.com/w/feed>) or  Follow us on feedly (<http://feedly.com/i/subscription/feed/https://blog.opendoor.com/rss>)

Opendoor (<https://www.opendoor.com>)

Get the app

 Download for Android (<https://play.google.com/store/apps/details?id=com.opendoor.buyerapp&hl=en>)

 Download for iOS (<https://itunes.apple.com/us/app/opendoor-homes-all-day-open/id1143708068?mt=8>)

Services

Sell your home (<https://www.opendoor.com>)

Buy a home (<https://www.opendoor.com/homes>)

Trade-in (<https://www.opendoor.com/trade-in>)

Home Value (<https://www.opendoor.com/home-value>)

How it works (<https://www.opendoor.com/how-it-works>)



Terms of use · Privacy policy (<https://www.opendoor.com/terms>) · Trust & Safety

(<https://www.opendoor.com/safety>)

 (<https://www.facebook.com/OpendoorHQ>)

 (<https://twitter.com/OpenDoor>)

 (<https://www.linkedin.com/company/opendoor-com>)